



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sede Bogotá Facultad de Ingeniería
Departamento de Sistemas e Industrial
Ingeniería de Software 1 (2016701)



Tutorial: Hola Mundo

Spring Boot + Hibernate + PostgreSQL

Framework: Spring Boot 4.0.6 via Spring Initializr

Lenguaje: Java 17

ORM: Hibernate (JPA)

Configuración: application.properties

Base de datos: PostgreSQL (Docker)

Integrantes:

Yohan Stiven Jimenez yjimenezh@unal.edu.co

Sebastian Fernando Gutiérrez Rojas segutierrezro@unal.edu.co

Bogotá, 12 de mayo del 2026

Índice

1. Introducción	3
2. Objetivos del tutorial	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Conceptos clave	3
4. Requisitos previos	3
4.1. Java 17 (JDK 17)	3
4.2. Maven	4
4.3. Docker y Docker Compose	5
4.4. Editor de texto	5
4.5. Navegador web	5
5. Arquitectura de la aplicación	6
5.1. Capas principales	6
5.2. Flujo de una petición	6
6. Paso 1 — Crear el proyecto con Spring Initializr	7
6.1. Dependencias a agregar	7
7. Paso 2 — Estructura del proyecto	8
7.1. Archivos generados automáticamente	9
7.2. Archivos principales de la aplicación	10
7.3. Archivos de configuración y recursos	10
7.4. Carpeta <code>static</code>	10
8. Paso 3 — Levantar la base de datos con Docker	12
9. Paso 4 — Configurar <code>application.properties</code>	13
10. Paso 5 — Clase principal	15
11. Paso 6 — Crear la entidad JPA	15
12. Paso 7 — Crear el repositorio	17
13. Paso 8 — Crear el servicio	19
14. Paso 9 — Crear el controlador REST	20
15. Paso 10 — Insertar el dato inicial	21
15.1. Opción A — Archivo <code>data.sql</code>	21
15.2. Opción B — <code>DataLoader</code> con <code>CommandLineRunner</code>	21
16. Paso 11 — Crear la interfaz web	22
16.1. Archivo <code>index.html</code>	22
16.2. Archivo <code>main.js</code>	23

17. Paso 12 — Ejecutar la aplicación	24
17.1. Verificar que PostgreSQL está activo	24
17.2. Ejecutar Spring Boot	24
17.3. Probar el endpoint desde la terminal	25
17.4. Ver la interfaz en el navegador	26
17.4.1. Pasos para visualizar la aplicación	26
18. Flujo completo de una petición	27
19. Conceptos REST relevantes	28

1. Introducción

2. Objetivos del tutorial

2.1. Objetivo general

Construir una aplicación web funcional con Java y Spring Boot capaz de conectarse a una base de datos PostgreSQL mediante Hibernate/JPA, exponiendo un endpoint REST verificable desde navegador o terminal.

2.2. Objetivos específicos

Al finalizar este tutorial serás capaz de:

- Crear un proyecto Spring Boot desde Spring Initializr.
- Configurar dependencias Maven para aplicaciones web y persistencia.
- Levantar una base de datos PostgreSQL utilizando Docker Compose.
- Configurar la conexión a la base de datos mediante application.properties.
- Implementar entidades JPA utilizando Hibernate.
- Aplicar una arquitectura en capas: Controller → Service → Repository.
- Crear endpoints REST con Spring Web.
- Consumir servicios backend desde una interfaz web sencilla usando JavaScript y fetch().
- Comprender el flujo completo de una petición HTTP dentro de una aplicación Spring Boot.

3. Conceptos clave

Antes de iniciar el desarrollo del proyecto, revisaremos de manera general las principales herramientas

4. Requisitos previos

4.1. Java 17 (JDK 17)

Java será el lenguaje principal de la aplicación. Para verificar si ya lo tienes instalado, abre una terminal y escribe:

```
java -version
```

Si la instalación es correcta, verás una salida similar a:

Tecnología	Descripción
JDK 17	Conjunto de herramientas que permite compilar y ejecutar programas Java. Sin él, el computador no sabría cómo interpretar el código que escribimos.
Maven	Gestiona las dependencias del proyecto y se encarga de compilarlo y ejecutarlo. Incluiremos Maven Wrapper para no requerir instalación global.
Spring Boot	Framework que simplifica el desarrollo de aplicaciones web con Java. Permite crear un servidor, conectarlo a una base de datos y responder peticiones con mínima configuración.
PostgreSQL	Base de datos relacional donde se almacenará la información de la aplicación.
Docker	Permite levantar servicios en contenedores listos para usar. Gracias a él tendremos PostgreSQL funcionando con un solo comando.
JPA / Hibernate	Tecnologías que conectan las clases Java con las tablas de la base de datos, permitiendo guardar y leer objetos sin escribir SQL directamente.

Cuadro 1: Herramientas y tecnologías utilizadas en el tutorial.

Herramienta	Versión mínima	Verificación
Java JDK	17	java -version
Maven	3.9+	mvn -version
Docker Desktop	24+	docker --version
IntelliJ IDEA	Cualquier CE	—
Navegador web	Cualquier moderno	—

Cuadro 2: Herramientas necesarias con sus versiones mínimas.

```
java version "17.0.12" 2024-07-16 LTS
Java(TM) SE Runtime Environment (build 17.0.12+8-LTS-286)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.12+8-LTS-286, mixed mode
, sharing)
```

Si no lo tienes instalado, descárgalo desde: <https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>

4.2. Maven

El proyecto incluye Maven Wrapper, una herramienta que permite ejecutar Maven sin necesidad de tenerlo instalado previamente en el computador, ya que se encarga de descargarlo automáticamente cuando se utiliza por primera vez. Su funcionamiento se

explicará más adelante durante la ejecución del proyecto. De igual manera, Maven también puede instalarse manualmente desde su página oficial: <https://maven.apache.org>

4.3. Docker y Docker Compose

Docker nos permitirá ejecutar PostgreSQL dentro de un contenedor aislado, sin necesidad de instalar manualmente la base de datos en el sistema operativo. Esto facilita la configuración del entorno de desarrollo y asegura que todos los integrantes del proyecto trabajen bajo la misma configuración.

En este tutorial utilizaremos Docker Compose para levantar PostgreSQL de forma rápida mediante un único comando. Para verificar que Docker y Docker Compose están correctamente instalados, ejecuta los siguientes comandos en la terminal:

```
docker --version
# Salida esperada:
Docker version 20.10.1, build 90ae1dd
```

```
docker-compose --version
# Salida esperada:
Docker Compose version v2.0.0
```

Si no lo tienes instalado, descarga Docker Desktop desde <https://www.docker.com/products/docker-desktop> (incluye Docker Compose en Windows y Mac). En Linux, sigue las instrucciones en <https://docs.docker.com/engine/install/>.



Advertencia

Asegúrate de que **Docker Desktop** esté en ejecución antes de continuar con el tutorial, ya que la base de datos se levanta como contenedor.

4.4. Editor de texto

Para desarrollar la aplicación necesitarás un editor o entorno de desarrollo que permita escribir y ejecutar código Java. En este tutorial se recomienda utilizar alguna de las siguientes opciones:

- **Visual Studio Code:** Editor ligero, gratuito y altamente configurable mediante extensiones. <https://code.visualstudio.com/>
- **IntelliJ IDEA Community:** Entorno de desarrollo especializado en Java, con herramientas avanzadas para proyectos Spring Boot y Maven. <https://www.jetbrains.com/idea/download/>

4.5. Navegador web

Para visualizar y probar la interfaz de la aplicación, puedes utilizar cualquier navegador web moderno compatible con tecnologías actuales, como Google Chrome, Mozilla Firefox, Microsoft Edge o Safari.

5. Arquitectura de la aplicación

Antes de escribir código es importante entender cómo están organizadas las capas de la aplicación y cómo fluye la información entre ellas.

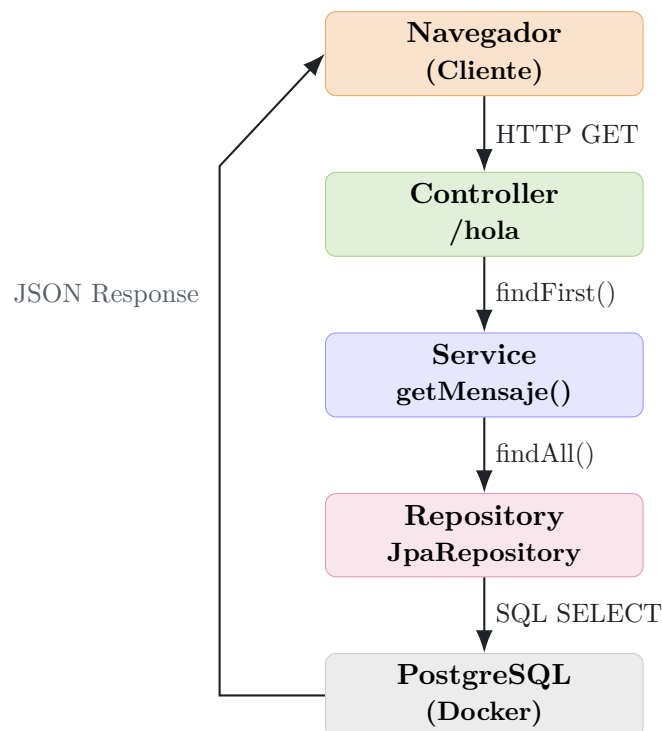
5.1. Capas principales

Capa	Responsabilidad
Controller	Recibe las peticiones HTTP del cliente (navegador) y devuelve la respuesta.
Service	Contiene la lógica de negocio; actúa como intermediario entre el controlador y el repositorio.
Repository	Maneja las operaciones con la base de datos usando JPA/Hibernate.
Model / Entity	Clase Java que representa una fila en la tabla SQL.

Cuadro 3: Capas de la arquitectura del proyecto.

5.2. Flujo de una petición

El siguiente diagrama muestra cómo fluye una petición GET `/hola` a través de las capas, de principio a fin:



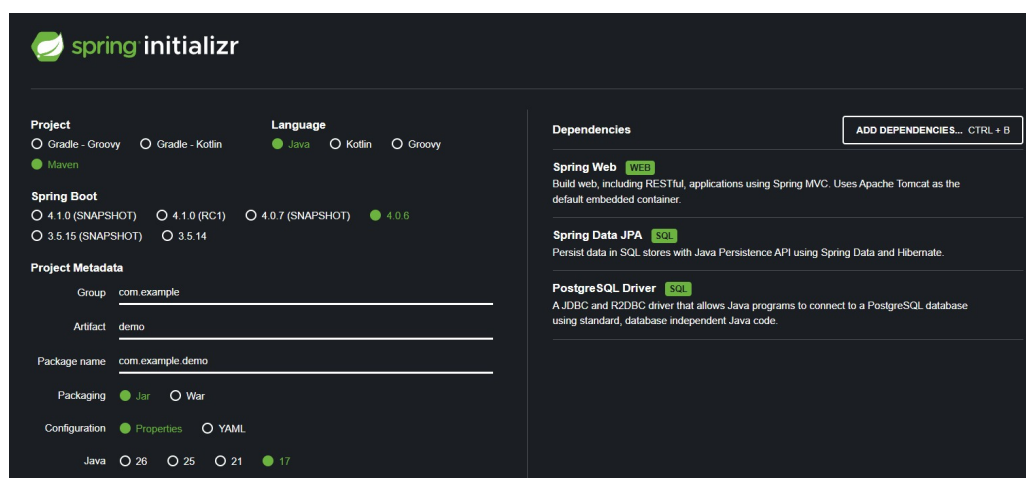
6. Paso 1 — Crear el proyecto con Spring Initializr

En la construcción del proyecto, generaremos la estructura inicial de la aplicación utilizando Spring Initializr. Esta herramienta oficial de Spring permite crear proyectos Spring Boot previamente configurados, incluyendo las dependencias necesarias y una organización base lista para comenzar el desarrollo.

Spring Initializr puede utilizarse tanto desde su sitio web como mediante la terminal. En este tutorial emplearemos inicialmente la versión web, ya que ofrece una interfaz más intuitiva y fácil de utilizar.

Paso 1

Abre el enlace: start.spring.io y genera la estructura base del proyecto con la configuración indicada a continuación.



The screenshot shows the Spring Initializr web interface. The 'Project' section has 'Maven' selected. The 'Language' section has 'Java' selected. The 'Spring Boot' section has '4.0.6' selected. The 'Project Metadata' section shows 'Group' as 'com.example', 'Artifact' as 'demo', and 'Package name' as 'com.example.demo'. The 'Packaging' section has 'Jar' selected. The 'Configuration' section has 'Properties' selected. The 'Dependencies' section has 'Spring Web', 'Spring Data JPA', and 'PostgreSQL Driver' selected. A button 'ADD DEPENDENCIES... CTRL + B' is visible in the top right of the dependencies section.

Figura 1: Formulario de Spring Initializr con la configuración del proyecto.

6.1. Dependencias a agregar

Haz clic en **ADD DEPENDENCIES** (atajo: Ctrl + B) y añade:

- **Spring Web** — para crear los endpoints REST.
- **Spring Data JPA** — incluye Hibernate como implementación de JPA.
- **PostgreSQL Driver** — conector JDBC para PostgreSQL.

Presiona **GENERATE**, descomprime el .zip en tu carpeta de trabajo y ábrelo en IntelliJ IDEA.

Consejo

También puedes importar directamente desde IntelliJ:
File → New → Project from Spring Initializr

Campo	Valor	Descripción
Project	Maven	Gestiona dependencias y compilación del proyecto.
Language	Java	Lenguaje principal utilizado por Spring Boot.
Spring Boot	4.0.6	Versión estable del framework utilizada en el proyecto.
Group	com.example	Identificador base de los paquetes del proyecto.
Artifact	demo	Nombre principal de la aplicación.
Package Name	com.example.demo	Paquete base donde se organizan las clases Java.
Packaging	Jar	Genera un archivo ejecutable de la aplicación.
Configuration	Properties	Formato de configuración usado por Spring Boot.
Java	17	Versión LTS recomendada por estabilidad y compatibilidad.

Cuadro 4: Configuración utilizada en Spring Initializr.

7. Paso 2 — Estructura del proyecto

Paso 2

Revisa la estructura generada y crea los paquetes adicionales necesarios.

Una vez importado, el proyecto se configura con la siguiente estructura de carpetas. Spring Boot la reconocerá automáticamente gracias al escaneo de componentes habilitado por `@SpringBootApplication`.

```
demo/
+-- .mvn/
|   +-- wrapper/
|       +-- maven-wrapper.properties
|
+-- src/
|   +-- main/
|       +-- java/
|           +-- com/
|               +-- example/
|                   +-- demo/
|                       +-- DemoApplication.java
|
|       +-- resources/
|           +-- application.properties
|           +-- static/
|           +-- templates/
|
```

```
|   +-- test/
|       +-- java/
|           +-- com/
|               +-- example/
|                   +-- demo/
|                       +-- DemoApplicationTests.java
|
+-- .gitattributes
+-- .gitignore
+-- HELP.md
+-- mvnw
+-- mvnw.cmd
+-- pom.xml
```

Cuando el proyecto es generado con Spring Initializr, Spring Boot crea automáticamente una estructura base de carpetas y archivos organizada bajo una convención estándar. Esta organización permite separar correctamente la lógica del programa, las configuraciones y los recursos utilizados por la aplicación.

Gracias a esta estructura, el código resulta más fácil de mantener, entender y escalar a medida que el proyecto crece.

7.1. Archivos generados automáticamente

Al crear el proyecto, Spring Initializr incluye algunos archivos esenciales que serán utilizados durante el desarrollo:

- **DemoApplication.java**: clase principal encargada de iniciar la aplicación Spring Boot.
- **application.properties**: archivo de configuración donde se definirán parámetros como la conexión a la base de datos y el puerto del servidor.
- **pom.xml**: archivo de Maven que administra las dependencias, plugins y configuración del proyecto.

A medida que avancemos en el tutorial, iremos agregando nuevas carpetas y archivos para separar cada responsabilidad de la aplicación.

7.2. Archivos principales de la aplicación

Tipo	Ruta relativa	Propósito
Entidad	model/Mensaje.java	Representa la tabla de la base de datos mediante una entidad JPA.
Repositorio	repository/MensajeRepository.java	Permite guardar y consultar información desde PostgreSQL.
Servicio	service/MensajeService.java	Contiene la lógica intermedia entre el controlador y el repositorio.
Controlador	controller/MensajeController.java	Expone endpoints REST que pueden ser consumidos desde el navegador.
Data Loader	com.example.demo/config/DataLoader.java	Inserta datos iniciales automáticamente al iniciar la aplicación.

Cuadro 5: Archivos principales utilizados en la arquitectura del proyecto.

7.3. Archivos de configuración y recursos

Archivo o carpeta	Propósito
application.properties	Define la configuración general del proyecto y la conexión a PostgreSQL.
data.sql	Permite insertar registros iniciales automáticamente en la base de datos.
static/index.html	Página principal mostrada en el navegador.
static/js/main.js	Contiene el código JavaScript encargado de comunicarse con el backend.

Cuadro 6: Archivos de configuración y recursos estáticos.

7.4. Carpeta **static**

Dentro del proyecto existe la carpeta:

```
src/main/resources/static
```

Spring Boot utiliza esta carpeta para almacenar todos los archivos accesibles directamente desde el navegador, como páginas HTML, hojas de estilo CSS, imágenes o archivos JavaScript.

Todo archivo ubicado allí puede abrirse escribiendo su ruta en el navegador. Por ejemplo:

```
http://localhost:8080/index.html
```

Esto es posible gracias a que Spring Boot incorpora un servidor web interno que detecta y publica automáticamente los recursos almacenados dentro de la carpeta static.

A medida que avancemos en el tutorial, agregaremos los siguientes archivos dentro del paquete principal:

```
demo/
+-- com.example.demo/
|   +-- DemoApplication.java
|   +-- model/
|       +-- Mensaje.java
|   +-- repository/
|       +-- MensajeRepository.java
|   +-- service/
|       +-- MensajeService.java
|   +-- controller/
|       +-- MensajeController.java
|   +-- config/
|       +-- DataLoader.java          (opcional)
|
+-- src/main/resources/
    +-- application.properties
    +-- data.sql                    (opcional)
    +-- static/
        +-- index.html
        +-- js/
            +-- main.js
```

Nota

Spring Initializr genera automáticamente archivos adicionales como mvnw, mvnw.cmd y la carpeta .mvn/, los cuales corresponden a Maven Wrapper y permiten ejecutar Maven sin necesidad de instalarlo globalmente en el sistema.

8. Paso 3 — Levantar la base de datos con Docker

Paso 3

Crea el archivo docker-compose.yml en la carpeta demo en el proyecto, y levanta el contenedor de PostgreSQL.

Crea el archivo docker-compose.yml con el siguiente contenido:

```
version: '3.8'

services:
  db:
    image: postgres:15

    environment:
      POSTGRES_DB: nopaindb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres

    ports:
      - "5432:5432"

    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```



Advertencia

Si estás trabajando en Linux, recuerda ejecutar los comandos con sudo. Además, asegúrate de que la terminal se encuentre ubicada en el mismo directorio donde está el archivo correspondiente. docker-compose.yml.

Si estás utilizando IntelliJ IDEA, puedes ejecutar los comandos directamente desde la terminal integrada del editor.

Para hacerlo:

1. Abre el proyecto en IntelliJ IDEA.
2. En la barra superior selecciona:

View → Tool Windows → Terminal

3. Se abrirá una terminal en la parte inferior del editor.
4. Verifica que la terminal esté ubicada en la carpeta raíz del proyecto, es decir, donde se encuentran archivos como pom.xml y docker-compose.yml.

Una vez ubicada en la ruta correcta, ejecuta:

```
docker compose up -d
```

Este comando levantará el contenedor de PostgreSQL en segundo plano. Si todo va bien, verás una salida similar a esta, indicando que el contenedor fue creado y arrancado:

```
[+] up 20/20
Image postgres:15      Pulled          37.1s
Network demo_default    Created         0.2s
Volume demo_db-data     Created         0.0s
Container demo-db-1     Started         5.3s
```

Verifica que el contenedor esté activo:

```
docker ps
```

Deberías ver el contenedor:

```
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
c04335eb53f0   postgres:15  "docker-entrypoint.s"    5 minutes ago Up 5 minutes  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  demo-db-1
```

9. Paso 4 — Configurar `application.properties`

Paso 4

Abre `src/main/resources/application.properties` y añade la siguiente configuración de la base de datos y JPA.

```
# -- Servidor -----
server.port=8080

# -- Base de datos (PostgreSQL) -----
spring.datasource.url=jdbc:postgresql://localhost:5432/nopaindb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

# -- JPA / Hibernate -----
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# -- SQL inicial -----
```

```
spring.sql.init.mode=always

# -- Nombre de la aplicacion -----
spring.application.name=demo
```

Descripción del contenido:

- `server.port=8080`
Define el puerto donde se ejecutará la aplicación Spring Boot. En este caso, el servidor estará disponible desde `http://localhost:8080`.
- `spring.datasource.url`
Indica la dirección de conexión hacia PostgreSQL, especificando el host, el puerto y el nombre de la base de datos que utilizará la aplicación.
- `spring.datasource.username`
Usuario utilizado para autenticarse en PostgreSQL.
- `spring.datasource.password`
Contraseña correspondiente al usuario configurado para la base de datos.
- `spring.datasource.driver-class-name`
Define el driver JDBC que Spring utilizará para comunicarse con PostgreSQL.
- `spring.jpa.database-platform`
Especifica el dialecto SQL que Hibernate debe utilizar para PostgreSQL.
- `spring.jpa.hibernate.ddl-auto=update`
Permite que Hibernate cree o actualice automáticamente las tablas de la base de datos según las entidades Java definidas en el proyecto.
- `spring.jpa.show-sql=true`
Muestra en consola las consultas SQL ejecutadas por Hibernate.
- `spring.jpa.properties.hibernate.format_sql=true`
Formatea las consultas SQL para que sean más legibles en la consola.
- `spring.sql.init.mode=always`
Indica que Spring debe ejecutar automáticamente archivos SQL de inicialización, como `data.sql`, cada vez que la aplicación inicia.
- `spring.application.name=demo`
Define el nombre interno de la aplicación dentro del ecosistema Spring Boot.

i Nota

Sobre `ddl-auto=update`: Hibernate leerá tus entidades y creará o actualizará las tablas automáticamente al iniciar la aplicación. En ambientes de producción usa `validate` o migraciones con Flyway/Liquibase.

10. Paso 5 — Clase principal

Cuando Spring Initializr genera el proyecto, crea automáticamente una clase principal encargada de iniciar toda la aplicación Spring Boot.

Este archivo ya existe dentro de la siguiente ruta:

```
src/main/java/com/example/demo/DemoApplication.java
```

Paso 5

Verifica que el archivo DemoApplication.java exista dentro del paquete principal del proyecto.

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

// Habilita autoconfiguracion + escaneo de componentes
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

La anotación @SpringBootApplication combina varias configuraciones internas de Spring Boot y habilita el escaneo automático de componentes (controladores, servicios y repositorios) dentro del mismo paquete y sus subpaquetes.

11. Paso 6 — Crear la entidad JPA

Ahora crearemos nuestra primera entidad JPA, llamada Mensaje.java.

Paso 6

Dentro del paquete principal com.example.demo, crea una nueva carpeta llamada model y dentro de ella el archivo Mensaje.java.

La ruta final debe quedar así:

```
src/main/java/com/example/demo/model/Mensaje.java
```

En IntelliJ IDEA puedes hacerlo así:

1. Expande la carpeta:


```
src/main/java/com/example/demo
```

2. Clic derecho sobre demo

3. Selecciona:

```
New -> Package
```

4. Escribe:

```
model
```

5. Luego clic derecho sobre model

6. Selecciona:

```
New -> Java Class
```

7. Escribe:

```
Mensaje
```

La clase debe contener el siguiente código:

```
package com.example.demo.model;

import jakarta.persistence.*;

@Entity
@Table(name = "mensaje")
public class Mensaje {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String texto;

    public Mensaje() {
    }

    public Mensaje(String texto) {
        this.texto = texto;
    }
}
```

```

    }

    public Long getId() {
        return id;
    }

    public String getTexto() {
        return texto;
    }

    public void setTexto(String texto) {
        this.texto = texto;
    }
}

```

A continuación se explican las anotaciones más importantes:

- `@Entity`: indica a JPA que esta clase representa una tabla en la base de datos.
- `@Table(name = "mensaje")`: especifica el nombre exacto de la tabla en PostgreSQL.
- `@Id`: marca el campo `id` como clave primaria.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)`: delega en la base de datos la generación automática del identificador (equivale a una columna `SERIAL` en PostgreSQL).
- `@Column(nullable = false)`: impone una restricción `NOT NULL` sobre la columna.

Nota

Con `ddl-auto=update`, la tabla `mensaje` se crea automáticamente al iniciar la aplicación si no existe. No es necesario escribir ningún `CREATE TABLE` en SQL.

12. Paso 7 — Crear el repositorio

El repositorio corresponde a la capa de acceso a datos de la aplicación. Su función es comunicarse directamente con la base de datos utilizando Spring Data JPA y Hibernate.

Paso 7

Dentro del paquete principal `com.example.demo`, crea una carpeta llamada `repository` y dentro de ella el archivo `MensajeRepository.java`.

La ruta final debe quedar así:

```
src/main/java/com.example.demo/repository/MensajeRepository.java
```

En IntelliJ IDEA:

1. Clic derecho sobre el paquete `com.example.demo`
2. Selecciona:

New -> Package

3. Escribe:

repository

4. Luego clic derecho sobre repository

5. Selecciona:

New -> Java Class

6. Escribe:

MensajeRepository

El archivo debe contener el siguiente código:

```
package com.example.demo.repository;

import com.example.demo.model.Mensaje;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface MensajeRepository
    extends JpaRepository<Mensaje, Long> {

}
```

Spring Data JPA provee automáticamente operaciones CRUD como:

- findAll()
- findById()
- save()
- deleteById()
- count()

Al extender `JpaRepository<Mensaje, Long>`, heredamos todas estas operaciones sin necesidad de escribir SQL manualmente.

13. Paso 8 — Crear el servicio

La capa de servicio actúa como intermediaria entre el controlador y el repositorio. Aquí se concentra la lógica de negocio de la aplicación.

Paso 8

Dentro del paquete principal `com.example.demo`, crea una carpeta llamada `service` y dentro de ella el archivo `MensajeService.java`.

La ruta final debe quedar así:

```
src/main/java/com.example.demo/service/MensajeService.java
```

El archivo debe contener el siguiente código:

```
package com.example.demo.service;

import com.example.demo.model.Mensaje;
import com.example.demo.repository.MensajeRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class MensajeService {

    private final MensajeRepository repository;

    public MensajeService(MensajeRepository repository) {
        this.repository = repository;
    }

    public List<Mensaje> findAll() {
        return repository.findAll();
    }

    public Mensaje save(Mensaje m) {
        return repository.save(m);
    }

    public Mensaje findFirst() {
        return repository.findAll().stream().findFirst().orElse(null);
    }
}
```

Esta clase define la lógica intermedia entre la base de datos y los endpoints REST.

Nota

Usamos inyección por constructor en lugar de `@Autowired` sobre atributos, ya que es la práctica recomendada por Spring: facilita las pruebas unitarias y evita problemas con referencias nulas.

14. Paso 9 — Crear el controlador REST

El controlador define los endpoints REST que pueden ser consumidos desde el navegador o desde cualquier cliente HTTP.

Paso 9

Dentro del paquete principal `com.example.demo`, crea una carpeta llamada `controller` y dentro de ella el archivo `MensajeController.java`.

La ruta final debe quedar así:

```
src/main/java/com.example.demo/controller/MensajeController.java
```

El archivo debe contener el siguiente código:

```
package com.example.demo.controller;

import com.example.demo.model.Mensaje;
import com.example.demo.service.MensajeService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/hola")
public class MensajeController {

    private final MensajeService service;

    public MensajeController(MensajeService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<Mensaje> getHola() {

        Mensaje m = service.findFirst();

        if (m == null) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok(m);
    }

    @PostMapping
    public ResponseEntity<Mensaje> createHola(
        @RequestBody Mensaje nuevo) {

        Mensaje saved = service.save(nuevo);

        return ResponseEntity.status(201).body(saved);
    }
}
```

Esta clase expone dos endpoints principales:

- GET /hola: retorna el primer mensaje almacenado.
- POST /hola: crea un nuevo mensaje en la base de datos.

Anotaciones importantes utilizadas:

- @RestController: indica que la clase funciona como controlador REST.
- @RequestMapping("/hola"): define la ruta base del controlador.
- @GetMapping: maneja solicitudes HTTP GET.
- @PostMapping: maneja solicitudes HTTP POST.
- @RequestBody: convierte automáticamente JSON en objetos Java.

15. Paso 10 — Insertar el dato inicial

Paso 10

Configura los datos iniciales para que la base de datos no quede vacía al arrancar la aplicación por primera vez.

Existen dos formas de insertar el mensaje “Hola Mundo” de manera automática. Elige la que mejor se adapte a tu proyecto.

15.1. Opción A — Archivo `data.sql`

Crea el archivo `src/main/resources/data.sql` que contiene lo siguiente:

```
INSERT INTO mensaje (texto) VALUES ('Hola Mundo');
```



Advertencia

Asegúrate de que el nombre de la tabla (`mensaje`) coincide exactamente con el valor definido en `@Table(name = "mensaje")` de la entidad. Además, ten en cuenta que este `INSERT` se ejecutará cada vez que se reinicie la aplicación, lo que puede generar registros duplicados.

15.2. Opción B — `DataLoader` con `CommandLineRunner`

Esta opción es más robusta: solo inserta el dato si la tabla está vacía, evitando duplicados al reiniciar la aplicación. Crea la carpeta `config` y dentro de ella el archivo `DataLoader.java` ubicado en la ruta `src/main/java/com.ejemplo.demo/config/DataLoader.java` con el siguiente contenido:

```
package com.gimnasio.demo.config;

import com.example.demo.model.Mensaje;
import com.example.demo.repository.MensajeRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DataLoader {

    @Bean
    CommandLineRunner initDatabase(MensajeRepository repo) {
        return args -> {
            if (repo.count() == 0) {           // Solo inserta si no hay
                datos
                repo.save(new Mensaje("Hola Mundo"));
                System.out.println("Dato inicial insertado: Hola Mundo
                ");
            } else {
                System.out.println("La base de datos ya tiene datos.
                Omitiendo carga inicial.");
            }
        };
    }
}
```

**Consejo**

Usa la **Opción B** si planeas expandir la lógica de inicialización en el futuro, ya que permite condiciones, ciclos y cualquier lógica Java antes de insertar los datos.

16. Paso 11 — Crear la interfaz web

Paso 11

Crea los archivos estáticos que el navegador mostrará al usuario.

Spring Boot sirve automáticamente los archivos ubicados en `src/main/resources/static/`. Cualquier archivo colocado allí es accesible desde el navegador sin configuración adicional.

La estructura de archivos que crearemos es la siguiente:

```
src/main/resources/static/
+-- index.html    <- Pagina principal
+-- js/
    +-- main.js   <- Logica que llama al backend
```

16.1. Archivo `index.html`

Crea el archivo `index.html` directamente en la carpeta `static/`:

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="utf-8" />
    <title>Hola Mundo - Spring Boot</title>
    <style>
      body { font-family: Arial, sans-serif;
        max-width: 600px; margin: 50px auto; padding: 20px; }
      button { padding: 10px 20px; font-size: 16px; cursor: pointer;
        background-color: #6DB33F; color: white;
        border: none; border-radius: 4px; }
      button:hover { background-color: #4B7A2B; }
      #resultado { margin-top: 20px; padding: 10px;
        border: 1px solid #ccc; background-color: #f9f9f9;
        min-height: 30px; border-radius: 4px; }
    </style>
  </head>
  <body>
    <h1>Hola Mundo &mdash; Spring Boot + PostgreSQL</h1>
    <button id="btn">Ver mensaje</button>
    <div id="resultado"></div>
    <script src="/js/main.js"></script>
  </body>
</html>
```

16.2. Archivo main.js

Crea la carpeta js/ dentro de static/ y luego el archivo main.js e inserta el siguiente código:

```
document.getElementById('btn').addEventListener('click', () => {
  fetch('/hola')
    .then(res => {
      if (!res.ok) throw new Error('Respuesta no OK: ' + res.status);
      return res.json();
    })
    .then(data => {
      document.getElementById('resultado').innerText =
        'ID: ' + data.id + ' - Texto: ' + data.texto;
    })
    .catch(err => {
      document.getElementById('resultado').innerText =
        'Error al obtener mensaje: ' + err.message;
    });
});
```

Nota

La función `fetch('/hola')` envía una solicitud HTTP al endpoint que definimos en el controlador. Cuando el servidor responde, el resultado JSON se muestra directamente en el `<div id=resultado>` de la página.

- `.then(res =>...)` verifica que la respuesta sea válida.
- `res.json()` convierte la respuesta al formato JSON que JavaScript puede interpretar.
- `.catch(err =>...)` captura cualquier error de red o del servidor y lo muestra al usuario.

17. Paso 12 — Ejecutar la aplicación

Paso 12

Arranca la aplicación y verifica que todos los componentes funcionan juntos.

Antes de probar los endpoints, debemos asegurarnos de que tanto PostgreSQL como la aplicación Spring Boot estén ejecutándose correctamente.

- Que el contenedor de PostgreSQL esté activo.
- Que Maven detecte correctamente el proyecto.
- Que Spring Boot inicie sin errores.
- Que el endpoint `/hola` responda correctamente.

Todos los comandos deben ejecutarse desde la raíz del proyecto, es decir, desde la carpeta donde se encuentra el archivo `pom.xml`.



Advertencia

Si estás trabajando en Linux, recuerda ejecutar los comandos con `sudo` cuando sea necesario.

17.1. Verificar que PostgreSQL está activo

Antes de ejecutar la aplicación, asegúrate de que el contenedor Docker esté corriendo:

```
docker ps
```

Si el contenedor no aparece en la lista, inícialo con:

```
docker compose up -d
```

17.2. Ejecutar Spring Boot

Desde la raíz del proyecto (donde está el archivo `pom.xml`), ejecuta:

```
# Linux / Mac
mvn spring-boot:run

# Windows con Maven Wrapper
mvnw.cmd spring-boot:run
```



Advertencia

En Windows PowerShell es necesario usar `.\` antes de `mvnw.cmd`, ya que PowerShell no ejecuta archivos del directorio actual automáticamente.

La primera ejecución puede tardar uno o dos minutos porque Maven descargará todas las dependencias. Las ejecuciones siguientes serán notablemente más rápidas. En la consola deberías ver algo como:

```

:: Spring Boot ::
(v4.0.0)

2026-05-12T12:11:43.859-05:00 INFO 12756 --- [demo] [
et\classes started by jackz in C:\Users\jackz\Documents\Ingesoft 1\tutorial\demo)
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] com.example.demo.DemoApplication : Starting DemoApplication using Java 17.0.12 with PID 12756 (C:\Users\jackz\Documen
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] com.example.demo.DemoApplication : No active profile set, falling back to 1 default profile: "default"
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 93 ms. Found 1 JPA repository interfac
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.21]
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 2125 ms
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] org.hibernate.orm.jpa : HHH000540: Processing PersistenceUnitInfo [name: default]
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] org.hibernate.orm.jpa : HHH000001: Hibernate ORM core version 7.2.12.Final
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@6636dd28
2026-05-12T12:11:43.868-05:00 INFO 12756 --- [demo] [
main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.

```

Figura 2: Imagen del comando

17.3. Probar el endpoint desde la terminal

Abre una nueva terminal y ejecuta:

```
curl.exe -i http://localhost:8080/hola
```

Respuesta esperada si el dato inicial fue insertado:

```
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Tue, 12 May 2026 17:24:51 GMT

{"texto":"Hola Mundo","id":1}
```

Si la tabla está vacía (no configuraste el DataLoader ni el data.sql), recibirás:

```
HTTP/1.1 404 Not Found
```

17.4. Ver la interfaz en el navegador

Una vez que la aplicación Spring Boot está en ejecución y el contenedor de PostgreSQL está activo, podemos interactuar con la interfaz web que construimos anteriormente.

17.4.1. Pasos para visualizar la aplicación

1. Abre tu navegador web preferido (Chrome, Firefox, Edge, Safari, etc.).
2. En la barra de direcciones, escribe la siguiente URL:

```
http://localhost:8080/index.html
```

3. Presiona Enter para cargar la página.
4. Una vez cargada la interfaz, haz clic en el botón Ver mensaje.
5. El sistema realizará una petición al endpoint GET /hola y mostrará el resultado.



Consejo

Si ves HikariPool-1 - Connection is not available, verifica que el contenedor de PostgreSQL esté corriendo con **docker ps**. Si ves Port 8080 already in use, cambia `server.port=8081` en `application.properties`.



Consejo

Error: Port 8080 already in use

Esto significa que el puerto 8080 ya está siendo utilizado por otro proceso. Soluciones posibles:

- Cambia el puerto en `application.properties`: `server.port=8081`
- O detén el proceso que está usando el puerto 8080 (ej: otra instancia de Spring Boot, Tomcat, etc.).
- En Windows: `netstat -ano | findstr :8080` para identificar el proceso.
- En Linux/Mac: `lsof -i :8080` para ver qué proceso lo ocupa.



Advertencia

La página carga pero el botón no muestra el mensaje

- Abre las herramientas de desarrollador (F12) y revisa la pestaña Consola.
- Busca errores como 404 Not Found o CORS policy.
- Verifica que el endpoint /hola esté respondiendo. Prueba desde otra terminal: `curl.exe -i http://localhost:8080/hola`
- Asegúrate de que la tabla no esté vacía (el DataLoader debe haber insertado el registro).

18. Flujo completo de una petición

Cuando el usuario interactúa con la interfaz web presionando el botón Ver mensaje, se desencadena una secuencia de eventos que atraviesa todas las capas de la arquitectura de la aplicación. Comprender este flujo es fundamental para internalizar cómo Spring Boot, Hibernate y PostgreSQL trabajan en conjunto.

1. El usuario presiona el botón en `index.html`.
2. JavaScript ejecuta `fetch('/hola')`.
3. El navegador envía un HTTP GET al servidor en el endpoint `/hola`.
4. `MensajeController` recibe la petición y llama a `service.findFirst()`.
5. `MensajeService` llama a `repository.findAll()`.
6. `MensajeRepository` ejecuta `SELECT * FROM mensaje` en PostgreSQL.
7. La BD devuelve `{id: 1, texto: "Hola Mundo"}`.
8. La respuesta viaja de vuelta por cada capa hasta el navegador como JSON.
9. JavaScript escribe el resultado en el `<div id=resultado>`.
10. El usuario ve ID: 1 - Texto: Hola Mundo en la pantalla.

Nota

El flujo completo de la aplicación demuestra cómo una simple interacción del usuario desencadena una secuencia de eventos que recorre todas las capas de la arquitectura: desde el controlador que recibe la petición HTTP, pasando por el servicio que contiene la lógica de negocio, hasta el repositorio que traduce las operaciones a consultas SQL mediante Hibernate, las ejecuta en PostgreSQL y devuelve el resultado inversamente hasta mostrarlo en pantalla. Así se consigue que un mensaje almacenado en la base de datos sea visible en la interfaz web, completando el ciclo de extremo a extremo de una aplicación moderna con Spring Boot.

19. Conceptos REST relevantes

Antes de dar el tutorial por terminado, es útil repasar algunos de los conceptos REST que utilizamos a lo largo de todo el proyecto:

Concepto	Descripción
Endpoint	URL que atiende peticiones HTTP. En este proyecto: /hola.
GET	Método HTTP que solicita información sin modificar el servidor. Lo usamos para leer el mensaje almacenado.
POST	Método HTTP que crea un nuevo recurso. Lo usamos para guardar un nuevo mensaje (opcional en este tutorial).
JSON	Formato de intercambio de datos entre cliente y servidor. Ejemplo: { <code>id</code> :1, <code>texto</code> :"Hola Mundo"}.
200 OK	La solicitud fue exitosa y el servidor devuelve el recurso solicitado.
201 Created	La solicitud fue exitosa y se creó un nuevo recurso.
404 Not Found	El recurso solicitado no existe (tabla vacía en nuestro caso).
500 Internal Server Error	Error inesperado en el servidor; revisa los logs de la consola.

Cuadro 7: Conceptos REST utilizados en el tutorial.

Nota

Los conceptos REST aplicados en este tutorial —endpoints, métodos HTTP (GET y POST), códigos de estado (200, 201, 404, 500) y el formato JSON— son los pilares fundamentales que permiten la comunicación entre el navegador (cliente) y el servidor Spring Boot. Comprender cada uno de ellos no solo es esencial para que la aplicación funcione correctamente, sino que también sienta las bases para construir APIs más complejas y robustas en el futuro.

20. Solución de problemas frecuentes

Error	Solución
Connection refused (5432)	Docker no está corriendo. Ejecuta <code>docker compose up -d</code> .
Table 'mensaje' doesn't exist	Verifica que <code>ddl-auto=update</code> esté en <code>application.properties</code> .
No qualifying bean of type 'MensajeRepository'	Asegúrate de que <code>@Repository</code> está presente y el paquete está dentro del paquete base de <code>@SpringBootApplication</code> .
Port 8080 already in use	Cambia <code>server.port=8081</code> o termina el proceso que ocupa el puerto 8080.
ClassNotFoundException: org.postgresql.Driver	Verifica que la dependencia <i>PostgreSQL Driver</i> está en <code>pom.xml</code> .
HTTP 404 al llamar /hola	La tabla está vacía. Asegúrate de haber configurado el <code>DataLoader</code> o el archivo <code>data.sql</code> .
HikariPool-1 Connection not available	El contenedor de PostgreSQL no está activo. Verifica con <code>docker ps</code> y reinicia si es necesario.
<code>application.properties</code> y <code>application.yml</code> coexisten	Si tienes ambos archivos, Spring Boot puede generar conflictos. Usa únicamente uno de los dos.

Cuadro 8: Errores comunes y sus soluciones.